HCR Dual-Track Project — Student Handbook

Instructor: Prof. Dr. Mohammed Aquil Mirza, PolyU, Hong Kong
Student Mentors: Mr. Ruan Haozhe, Mr. Jiang Suyu

Who this is for? Every student who joins one of the 10 teams. Read this end-to-end before your first lecture. It is the single upstream source for what we are building, how the work is divided, what you will be taught, and what you are expected to produce.

What this is. This is a handbook, not an engineering spec. The detailed contracts (IDL, topic table, worked-example fixtures) are released as separate documents after the first two lectures.

1. What We Are Building

One sentence: **a remote collaborative haircutting robot platform whose safety property is that a robot can only act on a plan that has passed through a simulated sandbox.**

This means the central piece of engineering is not the AI model that proposes a plan. It is the **layer in between**: the digital twin, the safety sandbox, and the message bus that connects them and the discipline that the same plan, the same sandbox, the same approval, are what eventually reach the real robot.

We will build two complete instances of this platform — Desktop and Web — that are deliberately different in every dimension **except their behavioral contract**. The point of building two is to prove that the platform is genuinely independent of host, transport, and rendering stack.

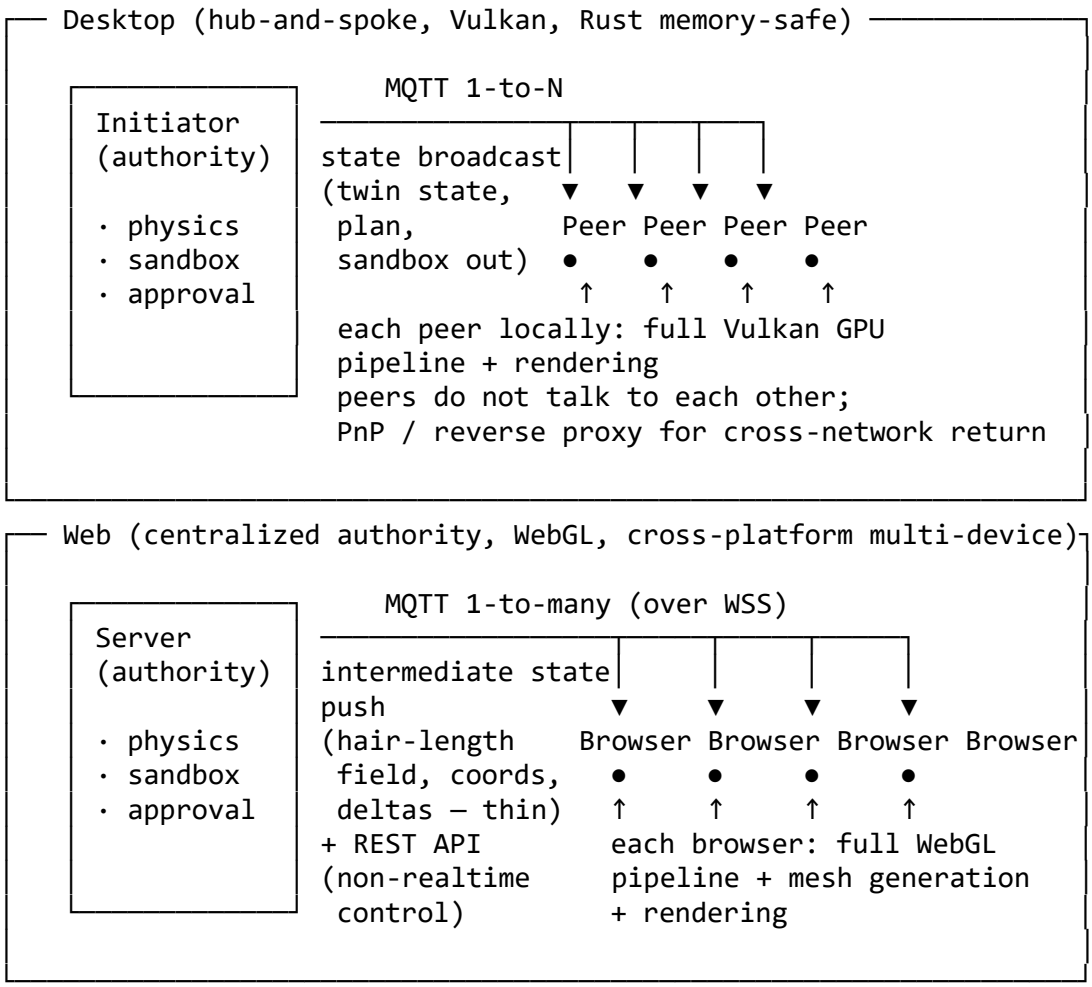
1.1 Concrete Objectives

By the end of this project, we will have demonstrated:

1. **Simulator-as-gatekeeper holds in both hosts.** Given the same sandbox semantics, whether you drive the system from a Rust desktop application or from a browser, the approve / reject / confirm behavior on identical inputs is identical.
2. **Three-way topic isomorphism.** sim-desktop ↔ sim-web ↔ real robot all consume the same topic graph and the same message schemas. The UI is just another subscriber on the bus.
3. **Desktop = infrastructure-free lightweight deployment.** An embedded MQTT broker + PnP / reverse proxy + N twins on one machine. No central server required.
4. **Web = dedicated server + cross-device.** REST + WSS MQTT dual protocol + WebGL on heterogeneous devices (laptop, tablet, mobile).
5. **One named access point named "Hotaru"** that makes transport swap (sim ↔ real) a non-event — flipping a config rather than rewriting code.
6. **Multi-robot safety-aware scheduling** running as a comparative experiment with reproducible data.
7. **Five worked examples** running end-to-end on both tracks: happy path / skin-distance violation / ambiguous command / out-of-order arrival / sensor impairment.

2. Dual-Track Architecture Overview

2.1 Visual Comparison



2.2-Dimensional Comparison

Dimension	Desktop	Web
Authority	Initiator (one of the participants)	Independent server
Topology	Hub-and-spoke, peers silently consume	Client-server
Cross-net reach	PnP / reverse proxy (lightweight deploy)	Independent IP/port + HTTP base
API form	MQTT-primary, REST rarely needed	Heavy REST API + WSS MQTT dual layer
MQTT impl	Native Rust (embedded)	JS (browser over WSS)
Client GPU stack	Vulkan (full performance)	WebGL (cross-platform)
Client duties	Rendering + mesh consumption	Full GPU pipeline (including mesh generation)
Sandbox location	Initiator	**Server only** (pure thin client)
Session model	MQTT 1-to-N, closed collaboration	MQTT 1-to-many, open subscription
Latency optimization	Light MQTT broadcast + zero-network local render	Light MQTT broadcast + delta increments
Typical deployment	On-site, transient, doctor house-calls	SaaS, training, remote demo
Client devices	Homogeneous Rust desktop	Heterogeneous (mobile/tablet/desktop browsers)
Security philosophy	**Trust the client** (Rust + initiator authority)	**Assume client is untrusted** (centralized sandbox, unbypassable)

2.3 The Only Thing Shared Is the Contract

Not shared: language, 3D stacks, UI framework, runtime implementation, hair-physics implementation, serialization details.

Shared (infrastructure level): the MQTT broker is the in-house hotaru_mqtt on both tracks — Desktop embeds it inside the initiator process, while Web runs it as a server-side broker that browsers reach over WSS. hotaru_mqtt is **not yet publicly released** and is being developed in ~/hotaru_mqtt/; teams use it as an internal dependency.

Only three things shared:

1. **Topic graph** (canonical string set)
2. **Message schemas** (JSON Schema / CBOR schema, language-neutral IDL)
3. **Behavioral specifications of the five worked examples** (given inputs → expected sequence of approve/reject/confirm)

Cross-track consistency = both tracks pass the same conformance test suite. **This is the executable definition of "the contract."** When you read "the contract" anywhere in this handbook, it means these three things and nothing else.

3. Team Structure

3.1 10 Teams × 2–3 People + ARB

We are organizing the work as 10 student teams. **Each team has 2–3 people and at least one ARB member** (typically the team lead). Each team owns one chapter of the final book and one slice of the system. Teams come in **twin pairs**: one team on the Desktop track and one team on the Web track build the same functional role on their respective host.

Desktop direction (Rust + wgpu + embedded hotaru_mqtt)

Team	Owned modules	Candidate innovation
D1 Shell	Desktop GUI + 3D viewport	Immediate-mode 3D + IME + realtime hair gizmo
D2 Twin	robot_twin + hair physics (Rust Verlet)	Native-Rust Verlet solver + trait-based physics-backend abstraction
D3 Brain	VLA + Scheduler (Rust)	Localized 5-factor scheduling decision visualization
D4 Safety	Sandbox + user_sim + emergency stop	Five-constraint stack + physical emergency stop channel
D5 Bus	embedded hotaru_mqtt + Hotaru-Rust + audit	PnP / reverse proxy lightweight deploy + cargo run nine-process

Web direction (TS/WASM + Akari + MQTT-over-WSS client to hotaru_mqtt)

Team	Owned modules	Candidate innovation
W1 Akari	Akari 3D + Islands shell	WebGL + four-island independent hydration
W2 Twin	in-browser twin + hair (WebGPU compute / WASM)	Browser-side Verlet GPU acceleration
W3 Brain	Web VLA bridge + Web scheduler	tool-call LLM + multi-operator collaboration
W4 Safety	server-side authoritative sandbox	Centralized sandbox + cross-device decision consistency
W5 Bus	MQTT-over-WSS + Hotaru-TS + Service Worker	REST + WSS dual protocol + offline buffering

ARB (Architecture Review Board) = the 10 team leads + 1 cross-track coordinator (11 people, part-time). The ARB exists to arbitrate three kinds of contract changes:

1. Topic graph changes (proposed by a Bus team)
2. Shared type changes (proposed by anyone, typically Bus / Safety)
3. Any request of the form "my team needs a topic only it uses" — **denied by default unless it is contract-isomorphic across both tracks**

If you are tempted to add a topic, change a field, or rename a type "just for your own piece," you are not allowed to do it inline. You write a one-page ARB proposal and bring it to the next sync.

3.2 Three Cross-Track Drift Risks (ARB Standing Agenda)

The reason the ARB exists is that the two tracks will, by default, drift apart in three specific places. The ARB watches these every week:

1. **D2 ↔ W2 hair behavior drift.** Both teams build their own hair-physics engine — D2 in Rust, W2 in WebGPU compute / WASM. Independent implementations of the same algorithm will produce slightly different results under the hair-length-floor check. We must specify "hair-length consistency" as a **testable tolerance** (e.g., "95% of vertices differ by < 0.1mm"), not as "roughly the same."

2. **D3 ↔ W3 scheduling formula drift.** The 5-factor priority formula must be **byte-for-byte identical** on both tracks. We publish it as a pure function plus a conformance fixture, and both implementations are tested against the same fixture.

3. **D5 ↔ W5 QoS semantics drift.** The native hotaru_mqtt broker and browser MQTT-over-WSS clients can exhibit subtle differences in reconnect / duplicate-delivery semantics. The effective semantics per topic are specified explicitly per topic — not as a QoS number alone.

If you are in one of the listed teams, **expect to be questioned about your pair's alignment status at every ARB sync**.

4. What Each Team Produces

Every team produces (a) a working implementation of its slice and (b) one chapter of the book. The chapter is the writeup; the working code is the evidence the chapter rests on. You cannot submit a chapter without the matching code.

4.1 Chapter ↔ Team Mapping (Twin-Pair Order):

The book has 10 chapters in twin-pair order, so the reader can compare paths side by side.

#	Chapter title (IGI Global format)	Team	Twin pair	Main thrust
1	Memory-Safe Desktop Shells for Collaborative Robotics: A Rust GUI and Vulkan Viewport Architecture	**D1 Shell**	↔ Ch2	Rust GUI selection, wgpu pipeline, Rust memory safety (trust the client)
2	Browser-Native Rendering for Cross-Device Robot Control: An Islands-Architecture WebGL Pipeline in Akari	**W1 Akari**	↔ Ch1	Akari scene graph, full WebGL pipeline + mesh generation, cross-device
3	Native-Rust Digital Twins for Robotic Haircutting: A Verlet Hair Solver and Trait-Based Physics-Backend Abstraction	**D2 Twin**	↔ Ch4	robot_twin, native-Rust Verlet hair solver, trait-based physics-backend abstraction, initiator authority + peers silently consume, PnP/reverse proxy

4	In-Browser Digital Twins With WebGPU Compute and WASM Verlet: An Intermediate-State Protocol for Cross-Device Decision Consistency	**W2 Twin**	↔ Ch3	in-browser twin, WebGPU compute / WASM Verlet, centralized authority + intermediate-state push, cross-device decision consistency
5	Safety-Aware Scheduling for Multi-Robot Platforms: A Five-Factor Priority Formula With Preemption and Fairness in Rust	**D3 Brain**	↔ Ch6	VLA interface, scripted policy, 5-factor priority, preemption/fairness (Rust implementation)
6	Vision-Language-Action Bridging and Multi-Operator Scheduling on the Web: A Tool-Call LLM Architecture for Collaborative Robotics	**W3 Brain**	↔ Ch5	Web VLA bridge, tool-call LLM, Web scheduler, multi-operator collaboration semantics
7	A Five-Constraint Sandbox for Robotic Haircutting: Approve/Reject/Confirm Semantics With a Physical Emergency-Stop Channel	**D4 Safety**	↔ Ch8	five-constraint stack (Rust), approve/reject/confirm, user_sim, physical emergency-stop channel
8	Server-Authoritative Safety in a Thin-Client Robotics Platform: Centralized Sandboxing for Cross-Device Decision Consistency	**W4 Safety**	↔ Ch7	five-constraint stack (server), sandbox only on server, decisions unbypassable, cross-device decision consistency
9	Embedded MQTT for Lightweight Robot Deployment: A Hotaru-Rust ABI With Audit Persistence and PnP/Reverse-Proxy Reachability	**D5 Bus**	↔ Ch10	embedded hotaru_mqtt broker (in-house, pre-release), Topic implementation on Rust side, Hotaru-Rust ABI, audit SQLite, PnP/reverse-proxy lightweight deploy
10	MQTT-over-WSS in the Browser: A Dual REST/WSS Control Plane With Hotaru-TS and Service-Worker Offline Buffering	**W5 Bus**	↔ Ch9	MQTT-over-WSS, REST control path, Topic implementation on TS side, Hotaru-TS ABI, Service Worker offline buffering

The twin-pair ordering is deliberate: a reader picks up the book and immediately sees, pair by pair, **how the same functional role is implemented on the Vulkan/Rust path versus the WebGL/TS path**. Your pair's conformance responsibility is visible in the table of contents — you cannot hide it.

4.2 You Own Your Chapter End-to-End

There is no separate editorial team. Each team is responsible for the full lifecycle of its own chapter — drafting, editing, formatting, citations, and final conformance to the book's chapter requirements. Your chapter must be submission-ready when you hand it over; the ARB performs only a compliance check, not a polish pass.

What this means in practice:

- **Your team self-edits** for grammar, tone, clarity, terminology, and figure/table style.
- **Your twin pair cross-reviews** each other's chapters as a mandatory step before ARB compliance review.
- **Your team owns citations** in APA 7.0 (the book's required style), reference-list correctness, and in-text citation completeness.
- **Your team owns figures, tables, and captions** — including alt text, numbering, and the required permission/source notes when reusing material.

Lightweight book-level coordination still exists, but it is minimal:

- **ARB writes only**: a one-page foreword and Appendices A–E (the executable contracts and traceability tables, see below).
- **Book preface / editor's introduction**: produced by the project lead as a single short piece, drawing from material every team has already written. No team writes "transitions" to other chapters — every chapter is standalone.
- **Final epilogue chapter** (cross-path conformance synthesis): written by the ARB after all 10 team chapters are submitted.

Appendices (ARB-maintained, frozen by lecture):

- Appendix A — shared IDL + type definitions (language-neutral canonical)
- Appendix B — full topic table (language-neutral canonical)
- Appendix C — five worked-example specifications + conformance fixtures
- Appendix D — chapter-by-chapter traceability table
- Appendix E — 10 teams / 5 twin pairs × modules × contract matrix

Appendices A–C are the executable contracts shared across twins. All 5 twin pairs align to these three appendices — not to each other. This is on purpose. It prevents the collaboration trap in which "whoever ships first becomes the de-facto standard."

4.3 Chapter Format (Required)

Each chapter must conform to the **IGI per-section word-count requirement** specified below. The required sections, in order:

1. **Title** (Unique and Descriptive) — concise, descriptive of the chapter's contribution. IGI Global format (long descriptive form with subtitle), as listed in §4.1.
2. **Abstract** (800–1100 characters) — single paragraph; concise summary of the chapter; main challenges and solutions; key results. No citations.
3. **Introduction** (600–1000 words) — problem statement and motivation; the chapter's objectives in the dual-track context; overview of approach.
4. **Related Works** (600–1400 words) — survey of relevant prior work; comparison with your approach. APA 7.0 in-text citations throughout.
5. **Implementation** (1400–2000 words) — architecture diagram of your slice; design decisions; algorithms; protocols; code-level details (synchronization, rendering, message routing); the contracts you expose to other chapters; code snippets for critical sections. This is where the **engineering substance** of your team's slice lives.
6. **Benchmark** (1000–1400 words) — test methodology; conformance-fixture results; performance metrics (throughput, latency, CPU usage, etc.); scalability tests; stress-testing results; tables and graphs.
7. **Discussion** (minimum 1000 words) — analysis of results; trade-offs in design choices; limitations and bottlenecks; comparison with alternative approaches; lessons learned about the dual-track contract.
8. **Conclusion and Future Work** (minimum 600 words) — summary of achievements; how objectives were met; potential improvements; future extensions.
9. **References** — **minimum 20 references in APA 7.0 style**; alphabetical; every in-text citation must appear here and vice versa. Include canonical project documentation, academic papers, and technical articles.

Per-section word counts are binding. A submission that falls outside any per-section range (above or below) fails the ARB compliance check and is returned for revision.

Summed chapter target: roughly **5,500–9,500 words per chapter** at typical density (the sum of the per-section minimums and maximums above). The full book targets ~170–300 pages plus appendices.

Figures and tables: numbered per chapter (Figure 1, Figure 2, …; Table 1, Table 2, …); each requires a caption and, when relevant, a source/permission note. Vector formats preferred for diagrams.

5. What We Will Teach You

There are four pre-recorded online lectures. Together they establish the shared foundation so that all 10 teams can write their own chapters independently and consistently.

5.1 Students and Roles

Student	Domain	Position in the 10-team system
Rs	Logic / Rust	Chapter direction of Core / Safety / Brain teams
Js	Hardware / Networks	Bus / Runtime teams + real-robot integration
JY	3D	D-Shell / D-Twin / W-Shell / W-Twin (four teams)

5.2 Course-Wide Strategy

Format: online pre-recorded lectures + separate Q&A sessions. Each lecture is **60 min** (excluding Q&A). Students can rewatch the recordings. **No live hands-on in the lecture.** All engineering implementation is done by your team between lectures.

Session	Lead	Topic	Who must attend
C1	**RS**	Thesis + dual-track architecture + domain model	All 10 teams must attend
C2	**JS**	Topic-isomorphic bus + worked examples	All 10 teams must attend
C3	**RS**	Sandbox + safety-aware scheduling + audit	Safety / Brain / Runtime / Core attend with high priority; others optional
C4	**JY**	3D path divergence + Hotaru runtime + writing convention	D-Shell / D-Twin / W-Shell / W-Twin / Integration high priority; **writing convention is mandatory for all teams**

RS takes two sessions because the domain model (C1) and safety/scheduling (C3) carry the heaviest argumentative density. **JS takes one** to transmit the entire bus contract in one shot. **JY takes one** to transmit the 3D dual-path divergence plus the book's writing convention (JY's chapters are the closing chapters of the book and are most sensitive to convention).

Each lecture has a fixed structure:

60 min Lead’s lecture (recorded)
N min Q&A (separate session, not counted in the 60 min)

Within 48 hours after each lecture: handout + recording + materials handed to teams are released, directly serving as upstream material for the 10 teams' chapter writing.

5.3 Lecture 1 (RS, 60 min) — Thesis + dual-track architecture + domain model

Audience: all 10 teams.

Outline (60 min)

- (10 min) The thesis: why it is not a better VLA model, but "the layer in between"
- (10 min) Simulator-as-gatekeeper: every approve message is the message consumed by the real robot
- (10 min) Dual-track motivation: Desktop "trust the client" / Web "client is untrusted"
- (15 min) The five core structs of haircut_core + serde IDL — the executable form of the cross-language contract

- (10 min) Audit schema: each decision is paired with the constraint result that produced it
- (5 min) Preview of next session: the bus contract

Materials handed to teams

- Handout PDF
- Shared IDL + type-definition markdown (seed for Appendix A)
- Audit-schema markdown
- Required-reading citations (key sections of the project's design documents)

Primary beneficiaries: **all 10 teams** (the domain model is the source 5 twin pairs align on). Concrete-landing chapters: **Ch 9 D-Bus** (Rust IDL implementation), **Ch 10 W-Bus** (TS IDL implementation). All twin pairs must, by the end of C1, verify their copy of the schema matches the other's.

5.4 Lecture 2 (JS, 60 min) — Topic-isomorphic bus + worked examples

Audience: all 10 teams.

Outline (60 min)

- (10 min) Bus, not Socket: the semantic gap between pub/sub and WebSocket
- (15 min) Full topic table: naming rules, variable segments, the reasoning behind retained vs non-retained
- (10 min) The engineering meaning of QoS 0/1/2 in robotic control; LWT and initiator-disconnect detection
- (15 min) The full flow of the five worked examples (happy / skin violation / ambiguous / reorder / impairment) as cross-path conformance tests
- (10 min) Preview of Desktop 1-to-N vs Web 1-to-many topology divergence

Materials handed to teams

- Full topic table (Appendix B draft)
- Message schemas in JSON Schema (refined Appendix A)
- Input / expected-output fixture set for the five worked examples (Appendix C)
- Index of relevant MQTT 5.0 spec sections

Primary beneficiaries: **Ch 9 D-Bus / Ch 10 W-Bus** (direct landing of topic implementation, Hotaru adapter, PnP/WSS dual protocol). The other 8 chapters consume the contract and must understand it. Twin pair Ch 9 ↔ Ch 10 must, by the end of C2, verify QoS and retained semantics match.

5.5 Lecture 3 (RS, 60 min) — Sandbox + safety-aware scheduling + audit

Audience: Safety / Brain / Runtime / Core high priority; others optional.

Outline (60 min)

- (15 min) Five-constraint stack: skin distance / no-cut zone / tool angle / hair-length floor / head-pose tolerance
- (10 min) Three states approve / reject / confirm; the fail-closed principle
- (15 min) 5-factor priority formula (stage / skin / confirm / safety / load); preemption thresholds; per-robot fairness
- (10 min) Audit log + replay + cross-version safety report
- (10 min) Hotaru named-access-point ABI overview: one handler shape, two roles

Materials handed to teams

- Pseudocode + threshold table for the five constraints
- Pure-function form of the 5-factor priority + conformance fixture
- audit-entry schema
- Hotaru trait-signature draft

Primary beneficiaries: **Ch 5 D-Brain / Ch 6 W-Brain** (scheduling formula must be byte-identical across both paths); **Ch 7 D-Safety / Ch 8 W-Safety** (constraint-stack implementation + three-state decisions); **Ch 9 D-Bus / Ch 10 W-Bus** (audit storage). Twin pairs (Ch5↔Ch6, Ch7↔Ch8) verify the scheduling-formula fixture and constraint-threshold table by the end of C3.

5.6 Lecture 4 (JY, 60 min) — 3D path divergence + Hotaru runtime + writing convention

Audience: D-Shell / D-Twin / W-Shell / W-Twin / Integration high priority; **writing convention is mandatory for all 10 teams**.

Outline (60 min)

- (10 min) Desktop path: wgpu / Vulkan single-machine pipeline; the coverage scope of Rust memory safety
- (10 min) Desktop path: initiator authority + peers silently consume + PnP / reverse proxy
- (10 min) Web path: Akari + WebGL full browser GPU pipeline + mesh generation
- (10 min) Web path: server pushes only intermediate-state fields + REST/WSS dual protocol + cross-device decision consistency
- (10 min) Cross-path conformance: behavior identical vs performance different vs deployment different
- (10 min) **Chapter-format conformance**: required section order (Title → References, IGI Global format), **IGI per-section word-count requirement** (Abstract 800–1100 chars; Introduction 600–1000; Related Works 600–1400; Implementation 1400–2000; Benchmark 1000–1400; Discussion ≥1000; Conclusion and Future Work ≥600; ≥20 APA 7.0 references), figure/table conventions, self-editing checklist, twin-pair cross-review process

Materials handed to teams

- Desktop / Web 3D-pipeline comparison table
- Chapter template (markdown, with section scaffolding)
- APA 7.0 citation style guide + figure/table examples
- Deliverable checklist for chapter submission
- Self-editing and twin-pair cross-review checklists

Primary beneficiaries: **Ch 1 D-Shell / Ch 2 W-Shell** (3D viewport + GPU pipeline comparison); **Ch 3 D-Twin / Ch 4 W-Twin** (digital twin + physics backend + render consumption); **Ch 9 D-Bus / Ch 10 W-Bus** (Hotaru ABI landing). Twin pairs (Ch1↔Ch2, Ch3↔Ch4) finalize the render-pipeline comparison table and cross-path behavior alignment items by the end of C4. **Chapter-format conformance guidance serves all 10 chapters.**

5.7 Four Lectures at a Glance

Dimension	C1 RS-1 Thesis + arch + domain model	C2 JS-1 Bus + WE	C3 RS-2 Sandbox + sched + audit	C4 JY-1 3D divergence + Hotaru + writing
Required	All 10 teams	All 10 teams	Brain / Safety / Bus (Four pairs + two Bus)	Shell / Twin / Bus (Six chapters + two Bus) + all teams for writing conv.
Primary twin-pair beneficiary	All 10 teams *Concrete:* Ch9 / Ch10	Ch9 D-Bus (schema alignment) Ch10 W-Bus	Ch5 D-Brain Ch6 W-Brain Ch7 D-Safety Ch8 W-Safety *(Audit Ch9 / Ch10)*	Ch1 D-Shell Ch2 W-Shell Ch3 D-Twin Ch4 W-Twin *(Hotaru Ch9 / Ch10)*
Materials + to teams	Shared IDL + constraint stack + audit schema	Topic table + dual-path comp. + WE	Priority fixture + JSON Schema + audit schema	Writing template + checklist
Twin-pair 5 pairs: post-class verification	All 5 pairs: schema agreement	Ch9 ↔ Ch10: QoS / retained agreement	Ch5 ↔ Ch6 Ch7 ↔ Ch8: formulas byte-id.	Ch1 ↔ Ch2 Ch3 ↔ Ch4: behavioral fixture agreement

5.8 Supplementary Videos (Optional, ~30 min each)

Beyond the four required lectures C1–C4, we will provide **optional supplementary videos** — short, fast tutorials of roughly 30 minutes each, **pre-recorded video format only (no live session, no scheduled Q&A)** — to fill in background that some students may lack. They are **not prerequisites**: the four required lectures give you everything strictly needed to start your chapter. Watch these videos only if your team needs them.

Topics we plan to provide:

- **Rust crash course** — ownership, borrowing, traits, async — for students new to Rust joining a Desktop-track team
- **MQTT basics** — pub/sub model, QoS 0/1/2, retained messages, last-will-and-testament — for any team that hasn't worked with message brokers
- **3D rendering fundamentals** — vertex/fragment pipeline, scene graphs, basic linear algebra — for non-3D students joining a Shell or Twin team
- **wgpu / Vulkan quick-start** — for D1 / D2 students unfamiliar with desktop GPU APIs
- **WebGL / WebGPU quick-start** — for W1 / W2 students unfamiliar with browser GPU APIs
- **Verlet hair physics** — algorithm walkthrough — for D2 / W2 teams
- **APA 7.0 citation style** — for students who haven't written in this style before

Additional video topics can be requested through the ARB. Lead time: roughly one week from request to release.

Supplementary videos are released on the same channel as C1–C4 recordings, but they ship **video + a one-page reference sheet only** — no contracts, no fixtures, no Q&A session. They cannot block downstream work.

6. What You Need to Do

6.1 The Chapter Production Flow

```
4 online recorded lectures (60 min lecture + separate Q&A each)
    │
    ▼
Each team receives: handout + recording + materials + chapter template
    │
    ▼
Inside the team:
  1. read upstream materials + Appendices A–C
  2. implement engineering evidence
  3. draft chapter in required format (2–3 weeks)
  4. self-edit:
     · grammar, tone, clarity
     · terminology aligned to Appendix B
     · APA 7.0 citations and reference list
     · figure/table numbering + captions
    │
    ▼
Twin-pair cross-review (mandatory, ~1 week):
  · pair reads each other's chapter
  · verify contract overlap is consistent on both sides
  · flag terminology drift, citation gaps, figure mismatches
    │
    ▼
ARB format-compliance check (pass/fail only — not a polish pass):
  · structure conforms to §4.3 (Title → References, IGI Global format)
  · every section is within its IGI per-section word-count range (§4.3)
  · ≥ 20 references; APA 7.0 reference list complete and consistent
  · Appendix A / B terminology used correctly
  Fail → returned to team for revision (no polish performed by ARB)
    │
    ▼
ARB integration: foreword, Appendices A–E, final epilogue chapter
    │
    ▼
Final book manuscript (~170–300 pages + appendices)
```

6.2 Operating Discipline (Read This Twice)

These are not suggestions. The project depends on them being followed.

1. **At the end of C1, haircut_core types + audit schema is frozen** and committed to the contract repo as read-only. All subsequent chapters take that version as input. Changes require ARB approval.
2. **At the end of C2, the topic table + worked-example fixtures are frozen.** If you discover, while writing your chapter, that you need a new topic, **you do not add it inline**. You write a one-page ARB proposal.
3. **Recording + handout + materials are released within 48 hours** of each lecture. If they are late, your team's onboarding cadence breaks — flag it loudly.
4. **Q&A sessions are not counted in the 60 min** of the recorded lecture. You are encouraged to record Q&A and cite it in your chapter writing.

5. **Chapter-delivery deliverable checklist** (from C4) is binding. The checklist enforces format conformance — structure, length, citation style, terminology, figures. **A submission that fails the checklist is returned to your team for revision. The ARB does not polish your chapter for you.** The checklist is not a formality.
6. **No live hands-on in the lectures.** This is deliberate. Recorded lecture + replayable + teams implement on their own → prevents lecture time from turning into "watching the instructor debug."
7. **Twin pairs talk to the contract appendices, not to each other.** If you and your twin team disagree on what a topic means, neither of you "wins" — you both go re-read Appendix B. If Appendix B is ambiguous, the fix is to file an ARB proposal to clarify Appendix B, not to settle it between yourselves.

6.3 What "Done" Looks Like

For your team, the project is done when:

- Your code implements the slice your team owns and runs on the canonical setup.
- Your slice passes the conformance fixtures for the worked examples it touches.
- Your chapter conforms to the required chapter structure (§4.3) and every section sits within its IGI per-section word-count range (Abstract 800–1100 chars; Introduction 600–1000; Related Works 600–1400; Implementation 1400–2000; Benchmark 1000–1400; Discussion ≥1000; Conclusion and Future Work ≥600), with ≥ 20 APA 7.0 references, and passes the deliverable checklist.
- Your team has self-edited: grammar, terminology aligned to Appendix B, APA 7.0 citations complete, figures/tables numbered with captions.
- Your twin pair has cross-reviewed each other's chapter and signed off on contract overlap (the specific items listed in §5.7 for your pair).
- Your chapter passes the ARB format-compliance check and is accepted into the manuscript.

7. Project Timeline

7.1 Phase 1 — Setup (~1 month)

- **All four lectures C1–C4 are recorded and released**; handout + recording + materials follow within 48 hours of each
- **At the end of C1**: haircut_core types + audit schema frozen and committed
- **At the end of C2**: topic table + message schemas + worked-example fixtures frozen
- **At the end of C4**: chapter template + deliverable checklist + self-editing checklist distributed to all 10 teams
- All 10 teams are officially onboarded, receive the IGI per-section word-count requirement (§4.3), and start chapter-first-draft writing (2–3 weeks)

7.2 Phase 2 — Build and Write (~10–16 weeks)

- All 10 teams draft, self-edit, and internally review their chapters against the chapter-format conformance checklist
- **Twin-pair cross-review (5 pairs)**: Ch1↔Ch2, Ch3↔Ch4, Ch5↔Ch6, Ch7↔Ch8, Ch9↔Ch10 — each pair reads and signs off on the other's chapter; the two chapters in each pair must describe consistent contract implementations; any disagreement falls back to Appendices A / B / C as authoritative
- All 10 teams submit publication-ready chapters; **submissions that fail the compliance check are returned to the team for revision (ARB does not polish)**
- ARB writes the foreword and the final epilogue chapter; ARB maintains Appendices A–E

- Engineering evidence for all five worked examples on both Desktop and Web paths is archived
- Cross-track conformance test suite passes
- Final book manuscript integrated (~170–300 pages + appendices)

7.3 Phase 3 — Real-Robot Integration

- Integrate with Client_RHCR (camera/calibration client) and begin transport swap from sim to real robot
- Each twin pair's worked-example fixtures re-run against the real-robot path; behavior must match

*** The End ***